# Equitransport

Equitransport was completed collaboratively by Zhengyi Yan and Ruoxuan Huang. We divided the work by package component, then integrated the modules into one reproducible workflow. Zhengyi led the initial project setup, data preparation, package structure, parts of the design report, documentations, and testing work. Ruoxuan contributed to other parts of the design report, the early test files and created several supporting modules. The GitHub commit history shows contributions from both members, including Zhengyi's commits for setup, data, access workflow, testing, CI/CD, documentation and packaging, and Ruoxuan's commits for testing, equity, plotting, utilities and pairwork documentation.

## Roles and Contributions

### Zhengyi Yan

Zhengyi set up the repository and Python package structure, added the initial data files, prepared parts of the design report and workflow diagram, and implemented the main GIS/public transport access workflow. This included developing and integrating the data-loading functions, the GTFS access calculation, weighted access support, GitHub Actions testing, README updates, demo script updates, Quarto documentation, changelog, license, and the final expanded test suite.

### Ruoxuan Huang

Ruoxuan contributed to other parts of the design report and several key supporting modules, including equity.py for equity analysis, plotting.py for visualisation functions, and utils.py for reusable helper functions. Ruoxuan also developed the initial pytest test file, helping establish the project's testing framework. In addition, she contributed to project documentation and communication materials, including written contents and structure in the readme file, and poster design and text preparation.

## Integration Approach

Our integration approach was to keep the package modular. Data loading, access calculation, equity metrics, plotting, and utility functions were separated into different modules under `src/equitransport`. This made it easier to combine individual contributions into a single workflow. After the core modules were created, we connected them through the public API in `__init__.py`, added an end-to-end demo script, and expanded the test suite to check normal cases, edge cases, error handling, CRS handling, and plotting outputs.

Although the original project brief focused on three main functions, `load_nzdep()`, `compute_access()`, and `equity_summary()`, our implementation evolved into a more modular structure during development. Instead of placing most of the logic inside a few large functions, we split the workflow into smaller data-loading, validation, access calculation, equity analysis, plotting, and utility functions. This made the package easier to test, debug, and maintain, especially because different parts of the workflow depended on different data sources and spatial analyses.

## What Worked Well

The modular structure worked well because each part of the project had a clear responsibility. Separating the package into data loading, access calculation, equity summary, plotting, and utility modules made it easier for us to work on different parts without constantly editing the same files. This reduced the chance of GitHub conflicts and made the package easier to test, debug, and document.

Planning our roles before starting also helped because we had a clearer idea of who was responsible for each functionality. Once the main function inputs and outputs were agreed, it became easier to integrate the separate modules into one complete workflow. The final demo maps and summary outputs also helped us check that the package was answering the original project question.

## What Went Wrong or Was Difficult

The main difficulties were data handling, GitHub coordination, and packaging/versioning. The project required several different data sources, including NZDep, SA1/SA2 boundaries, and GTFS files, so making the workflow reproducible took more work than expected. We also had to manage GitHub updates and integration carefully, including pulling from main and resolving changes across commits. Near the end, we had to improve versioning, documentation, and test coverage as we realised what we had was not sufficient, especially the number of tests we had at the time. Writing tests was also challenging for us because we had a lot of different modules and that led to a large number of different unit tests and integration tests, which took a while to implement.